Mayanna–HashMin Primitives:
Simple Hash-Based Zero-Knowledge Predicate Proofs for Small Credential Claims

Mayank Muthanna
mayankmuthanna.m@gmail.com

July 20, 2026

Abstract

Many digital services need to verify simple facts about a person, such as whether a user is over 18, whether a score is above a threshold or whether a membership level is high enough. Full zero-knowledge proof systems can support these checks, but they often require complex cryptographic machinery that can be difficult to implement, audit and deploy in lightweight credential systems.

This paper presents Mayanna–HashMin Primitives, a deliberately simple construction for proving predicates over small credential attributes using hash commitments, Merkle trees, digital signatures and MPC-in-the-head. An issuer commits to the bits of an attribute, signs the Merkle root and gives the credential witness to the holder. Later, the holder proves that the committed value satisfies a public predicate by privately evaluating a Boolean circuit through three simulated MPC parties.

The circuit checks both that the private bits correctly open the issuer-authenticated commitments and that the requested predicate is true. The holder commits to all three simulated views, and the verifier opens two views selected by a challenge. Repeating this process reduces the probability that a false proof is accepted. The Fiat–Shamir transform can make the proof non-interactive.

The verifier learns the public predicate and whether it is true, but does not learn the credential bits, the comparison prefix, the decisive bit or the commitment randomness. The construction is intended as a readable and audit-oriented zero-knowledge primitive for small credential predicates, not as a replacement for general-purpose zero-knowledge systems.

1 Introduction

A website that needs to check whether a user is old enough to access a service usually does not need the user's exact date of birth. It only needs to know whether a simple condition is true, such as:

$$\text{age} \geq 18.$$

The usual real-world solution is much more invasive. The user is made to upload an identity document, share a birth date or send an entire credential. This reveals more information than the verifier needs. It also creates storage, security and compliance risks for the verifier.

Cryptographic systems already exist for proving statements while hiding private information. General zero-knowledge proof systems, range proofs and anonymous credential systems can prove threshold claims without revealing the exact value. However, these systems can be difficult to

implement and audit. Many rely on elliptic-curve arithmetic, pairing-based cryptography, polynomial commitments, trusted setup ceremonies or large cryptographic libraries.

This paper asks a narrower question:

> Can we build a readable zero-knowledge primitive for small credential predicates using hash commitments, Boolean circuits and MPC-in-the-head?

Our answer is Mayanna–HashMin Primitives.

The construction is designed for small attributes such as age, score, level, rank or eligibility number. The issuer writes the value in binary, commits to each bit using fresh randomness, places the commitments in a Merkle tree and signs the root.

To prove a predicate, the holder does not open any credential bit. Instead, the holder evaluates a Boolean circuit that checks two things.

First, the circuit checks that the private bits and private randomness correctly open the public commitments authenticated by the issuer.

Second, the circuit checks that the private value satisfies the requested predicate.

The holder evaluates this circuit using MPC-in-the-head. Three virtual parties receive secret shares of the private witness and jointly evaluate the circuit inside the holder's local computation. The holder commits to the three simulated views. The verifier then asks to inspect two views and checks that they are correct and consistent.

1.1 What This Paper Claims

This paper makes four modest claims.

1. Mayanna–HashMin gives a hash-oriented primitive for proving small predicates over issuer-authenticated credential attributes.
2. The construction reveals no credential bits, comparison prefix or decisive position to the verifier.
3. Predicate soundness follows from commitment binding, issuer signature unforgeability and repeated MPC-in-the-head challenges.
4. The proof layer can be implemented using hashes and Boolean operations without requiring a general-purpose SNARK or pairing-based proof system.

1.2 What This Paper Does Not Claim

This paper does not claim to invent MPC-in-the-head. The connection between secure multiparty computation and zero-knowledge was introduced by Ishai, Kushilevitz, Ostrovsky and Sahai. Practical three-party constructions for Boolean circuits were later developed in systems such as ZKBoo.

The contribution here is the use of this method for issuer-authenticated hash commitments over small credential attributes.

This paper also does not claim that the construction is succinct. MPC-in-the-head proofs are usually larger than SNARK proofs because proof size grows with the circuit and the number of repetitions.

Finally, this paper defines a primitive rather than a complete credential protocol. Replay protection, revocation, expiry, verifier policy and implementation benchmarks are separate protocol-layer questions.

1.3 Intended Use Cases

Mayanna–HashMin is intended for claims such as:

- proving that age is at least 18;
- proving that a score is above a cutoff;
- proving that a membership level is high enough;
- proving that a numeric value lies inside a public range;
- proving a small combination of credential conditions.

The construction is not intended for large private programs or applications where the smallest possible proof size is required.

2 Background

This section reviews the building blocks used by Mayanna–HashMin.

2.1 Hash Functions

A cryptographic hash function takes an input and returns a fixed-size output. For example, SHA-256 returns a 256-bit output.

Two properties matter here.

First, it should be difficult to recover the hidden input from the output when the input contains enough fresh randomness.

Second, it should be difficult to find two different inputs with the same output. This is collision resistance.

Mayanna–HashMin uses hashes for credential commitments, Merkle trees, simulated-view commitments and Fiat–Shamir challenges.

2.2 Hash Commitments

For a credential bit b_i, where the bit is either 0 or 1, the issuer chooses fresh independent randomness r_i and computes:

$$C_i = H(\texttt{Mayanna-HashMin-Leaf} \parallel \text{schema} \parallel i \parallel b_i \parallel r_i).$$

The position and schema are included so that a commitment cannot be moved to another bit position or interpreted under another credential format.

Every bit must use fresh randomness. Randomness must not be reused across positions or credentials.

Without randomness, anyone could hash 0 and hash 1 and compare the outputs. With sufficiently large randomness, the commitment hides the bit.

2.3 Merkle Trees

A Merkle tree compresses all bit commitments into one root hash.

The ordered commitments become the leaves of the tree. The issuer signs the root together with the schema and bit length.

During verification, the holder provides the ordered commitment list. The verifier recomputes the Merkle root and checks the issuer signature. The commitments themselves do not reveal their bits when the randomness is strong.

2.4 Digital Signatures

The issuer signs the credential root.

The verifier checks the signature before trusting the credential commitments. This separates two questions:

- Did a trusted issuer authenticate this credential?
- Do the hidden committed bits satisfy the requested predicate?

The signature answers the first question. The MPC-in-the-head proof answers the second.

2.5 Boolean Circuits

A Boolean circuit is a program made from wires carrying bits and gates operating on those bits.

Mayanna–HashMin uses XOR, AND and NOT gates. These gates are enough to build comparisons, equality checks, range checks and hash functions.

The circuit has public and private inputs.

The public inputs include the bit commitments and the requested threshold.

The private inputs include the credential bits and the randomness values used to create the commitments.

The circuit outputs 1 only when all commitments open correctly and the predicate is true.

2.6 MPC-in-the-Head

Secure multiparty computation normally lets several parties jointly evaluate a function without revealing their private inputs.

MPC-in-the-head uses this idea without separate machines. The holder locally simulates three virtual parties.

Every private input bit is split into three XOR shares:

$$b = b_0 \,\mathrm{XOR}\, b_1 \,\mathrm{XOR}\, b_2.$$

The first two shares are random. The third is chosen so that the three shares reconstruct the original bit.

Any two shares look random without the third share. This lets the verifier inspect two simulated views without learning the private input.

The holder commits to all three views before learning which two must be opened.

3 The Mayanna–HashMin Construction

3.1 Setting

There are three real parties.

- The issuer creates and signs credentials.
- The holder owns a credential and wants to prove a claim.
- The verifier checks the issuer signature and the proof.

The three real parties must not be confused with the three virtual MPC parties. The virtual parties exist only inside the holder's proof-generation algorithm.

The holder has a private attribute value x. The verifier provides a public predicate, such as:

$$x \geq T.$$

The value is represented using a fixed number of bits.

3.2 Credential Creation

To issue a credential for a value x, the issuer performs the following steps.

1. Convert x into binary using a fixed number of bits.
2. For each bit position, sample fresh independent randomness.
3. Create one salted hash commitment per bit.
4. Place the ordered commitments into a Merkle tree.
5. Sign the Merkle root together with the schema and bit length.
6. Give the holder the bits, the randomness values, the commitments and the issuer signature.

For example, if:

$$x = 13 = 1101_2,$$

the issuer commits separately to 1, 1, 0 and 1.

3.3 The Predicate Circuit

The proof circuit checks the relationship between the public commitments and the private witness.

For every bit position, the circuit:

1. receives the bit and randomness as private inputs;
2. recomputes the salted hash commitment;
3. compares the computed output with the public commitment.

Every commitment check must succeed.

The circuit then reconstructs the logical value represented by the private bits and checks the requested predicate.

For a greater-than-or-equal comparison, the circuit reads the bits from the most significant position to the least significant position.

It remembers whether all earlier bits were equal and whether the private value has already become greater than the threshold.

At the end, the circuit outputs 1 if the private value is greater than the threshold or exactly equal to it.

The comparison is evaluated inside the circuit. The verifier does not learn which position decided the result.

3.4 Creating the Simulated Views

For every repetition, the holder performs the following steps.

1. Secret-share every private input bit among three virtual parties.
2. Give every virtual party the same public inputs.
3. Give every virtual party an independent random tape.
4. Evaluate the complete Boolean circuit in three simulated branches.
5. Record one view for every virtual party.

A view contains the party's private input shares, random tape, wire values, AND-gate data and output share.

3.5 Gate Evaluation

XOR gates are evaluated locally. Every virtual party XORs its two input shares.

For a NOT gate, one fixed virtual party flips its share and the other two leave their shares unchanged.

AND gates require fresh random masks.

Suppose the parties hold shares x_0, x_1, x_2 and y_0, y_1, y_2. For party i, let $i + 1$ mean the next party, wrapping around after party 2.

One form of the Ishai-style local AND calculation is:

$$z_i = (x_i \text{ AND } y_i) \text{ XOR } (x_{i+1} \text{ AND } y_i) \\ \text{XOR } (x_i \text{ AND } y_{i+1}) \text{ XOR } R_i \text{ XOR } R_{i+1}.$$

The mask bits come from independent random tapes and must be fresh for every AND gate.

When all three output shares are XORed together, the masks cancel and the result is the AND of the original hidden input bits.

3.6 Committing to the Views

After evaluating the circuit, the holder commits separately to the three views.

The commitment includes:

- the public statement;
- the repetition number;
- the virtual party number;
- fresh commitment randomness;
- the complete serialized view.

The holder must commit before learning which views will be opened.

3.7 Challenge and Opening

There are three possible challenges.

The verifier may request:

- views 0 and 1;
- views 1 and 2;
- views 2 and 0.

The holder opens the requested two views.

The verifier checks:

- both view commitments;
- all XOR and NOT transitions;
- the AND-gate calculations;
- consistency between the two opened views;
- the output shares belonging to those views.

All three output shares are included in the proof. The verifier checks that they reconstruct to 1.

3.8 Repeating the Proof

One repetition has a soundness error of at most two thirds. A dishonest holder may prepare a false execution that passes two challenge choices but fails the third.

The process is therefore repeated with fresh randomness.

For s repetitions, the basic soundness error is:

$$\left(\frac{2}{3}\right)^{s}.$$

For example, 137 repetitions give roughly 80 bits of soundness, while 219 repetitions give roughly 128 bits.

3.9 Making the Proof Non-Interactive

The Fiat–Shamir transform replaces the verifier's random challenge with a hash-derived challenge.

The holder first creates the view commitments for every repetition.

The holder then hashes:

- the complete public statement;
- the circuit identifier;
- the credential root;
- the public predicate;
- every view commitment;
- every output share.

This produces one challenge choice for every repetition.

All challenges must be derived together after all view commitments have been fixed. This prevents the holder from repeatedly discarding unfavorable rounds and generating replacements.

3.10 Proof Verification

The verifier checks the proof as follows.

1. Verify the issuer signature.
2. Recompute the Merkle root from the ordered commitments.
3. Reconstruct the public predicate circuit.
4. Recompute the Fiat–Shamir challenges.
5. For every repetition, verify the two required view openings.
6. Check every available gate transition and cross-view consistency condition.
7. Check that all three output shares reconstruct to 1.
8. Accept only if every check succeeds.

3.11 Example

Suppose the holder has:

$$x = 13 = 1101_2$$

and wants to prove:

$$x \geq 8.$$

The circuit privately checks that the four value bits open the four issuer-authenticated commitments.

It then compares:

$$1101_2$$

with:

$$1000_2.$$

The circuit output is 1.

The verifier learns that the committed value is at least 8.

The verifier does not learn that the second bit decided the comparison, and does not learn any of the four private bits.

4 Correctness and Security

4.1 Correctness

Correctness means that an honest proof verifies.

Theorem 1 (Correctness)**.** *If the issuer honestly creates the credential, the holder uses the correct openings, the predicate is true and the proof is generated honestly, then the verifier accepts.*

Proof. Every private bit correctly opens its public commitment. The predicate circuit therefore receives the issued value and outputs 1. The three-party evaluation reconstructs the same output. Every opened view is valid and consistent, so the verifier accepts.

4.2 Commitment Binding

Theorem 2 (Binding)**.** *If the hash function is collision resistant, a holder cannot open the same authenticated bit commitment as both 0 and 1.*

Proof. Opening the same commitment to two different bits requires two different hash inputs with the same output. This is a hash collision.

4.3 Issuer Authenticity

Theorem 3 (Issuer Authenticity)**.** *If the signature scheme is unforgeable, a holder cannot create a new credential root that appears to be signed by the issuer unless the issuer signed it.*

Proof. The verifier accepts only after checking the signature on the credential root and schema. Creating a different signed root requires forging the issuer signature.

4.4 Predicate Soundness

Theorem 4 (Predicate Soundness)**.** *Assume that the issuer signature is unforgeable, the hash commitments and view commitments are binding and the Fiat–Shamir hash is modeled as a random oracle. If the committed value does not satisfy the predicate, a malicious holder can make the verifier accept only with probability bounded by the MPC-in-the-head soundness error, except with negligible probability.*

Proof. The issuer signature fixes the credential root. The Merkle root fixes the ordered commitments, and commitment binding fixes their bit openings.

If those bits do not satisfy the predicate, a correct circuit execution outputs 0.

To claim output 1, the holder must create inconsistent simulated views. In each repetition, at least one of the three challenge pairs exposes the inconsistency. The chance of avoiding that pair is at most two thirds.

After s independent repetitions, the chance of passing all false rounds is at most $(2/3)^s$.

Changing a view after seeing the challenge would require breaking the binding of its commitment.

4.5 Zero Knowledge

Theorem 5 (Zero Knowledge, Informal)**.** *Under the privacy of the three-party MPC decomposition, the hiding of unopened view commitments and the random-oracle analysis of Fiat–Shamir, the proof reveals no private credential information beyond what follows from the public statement and the truth of the predicate.*

Proof intuition. The verifier sees only two of the three simulated views.

The three-party decomposition is designed so that two views can be simulated without knowing the private input, as long as the public output is known. The missing third view contains the information needed to complete the secret sharing.

The unopened view remains hidden behind its commitment.

Therefore, the verifier does not learn the credential bits or commitment randomness from the opened views.

4.6 Hiding of Public Commitments

The zero-knowledge proof does not replace the hiding requirement of the credential commitments.

For an unopened bit commitment, the verifier sees a hash containing the bit and fresh randomness.

If the randomness is large and unpredictable, the verifier cannot test whether the committed bit is 0 or 1.

If randomness is weak or reused, the commitments may leak information even when the MPC proof is correctly implemented.

5 Privacy and Leakage

Mayanna–HashMin does not reveal credential bits.

For a successful proof, the verifier learns:

- the issuer identity or public key;
- the signed credential root;
- the public commitments;
- the credential schema and bit length;
- the public predicate;
- that the predicate is true;
- the proof parameters and proof size.

The verifier does not learn:

- the exact credential value;
- any individual credential bit;
- the matching comparison prefix;
- the decisive comparison position;
- the commitment randomness;
- the unopened simulated view.

5.1 Information Implied by the Predicate

Zero knowledge cannot hide information that logically follows from the public claim.

A proof that age is at least 18 necessarily reveals that fact.

Several different predicate requests can narrow the possible value. For example, proving both age at least 18 and age less than 20 reveals a small interval.

This is information implied by the chosen predicates, not leakage from the proof transcript.

5.2 Linkability

If the same credential root is shown repeatedly, verifiers may link those presentations.

MPC-in-the-head hides the witness, but it does not automatically randomize a persistent credential commitment.

Unlinkable presentation requires an additional mechanism and is outside this primitive.

6 Efficiency

Let n be the number of credential bits and let s be the number of repetitions.

6.1 Credential Size

The issuer creates one hash commitment per bit, so there are n Merkle leaves.

The Merkle tree contains $O(n)$ nodes.

The holder stores the bits, one randomness value per bit, the commitments and the issuer signature.

6.2 Circuit Size

The comparison part of the circuit grows linearly with the bit length.

The expensive part is checking the commitment openings. The circuit must evaluate the selected hash function for every committed bit.

For small attributes, this may still be practical, but the hash circuits are expected to dominate the computation.

6.3 Proof Size

A straightforward proof includes three view commitments and two opened views for every repetition.

The proof size grows with:

- the repetition count;
- the private input length;
- the number of circuit gates.

The construction is therefore not succinct.

Seed expansion and view compression can reduce proof size in future implementations.

6.4 Computation

The issuer computes $O(n)$ commitments, builds a Merkle tree and creates one signature.

The holder evaluates the complete circuit in three simulated branches for every repetition.

The verifier checks two branches per repetition.

XOR and NOT gates are inexpensive. AND gates, hash circuits and view hashing dominate the cost.

7 Comparison to Existing Approaches

Mayanna–HashMin should be understood as a readable credential primitive built from known zero-knowledge techniques.

Approach	Strength	Mayanna–HashMin Difference
SNARKs	Succinct and expressive	Avoids pairing-based proof machinery, but produces larger proofs.
Bulletproofs	Strong range proofs without trusted setup	Uses Boolean circuits and hashes rather than elliptic-curve inner-product arguments.
IKOS and ZKBoo	General MPC-in-the-head zero knowledge	Specializes the method to issuer-authenticated credential commitments and small predicates.
HashWires-style systems	Optimized hash-based range proofs	Hides the complete comparison structure and supports Boolean predicates, but is less compact.
Selective-disclosure credentials	Practical disclosure of signed fields	Proves a predicate without revealing the underlying field.

The main goal is not to make the smallest proof. It is to make the proved relationship clear:

> The hidden value must open the issuer-authenticated commitments, and the same hidden value must make the public predicate circuit output 1.

8 Limitations

Mayanna–HashMin has several important limitations.

1. It is not succinct. Proof size grows with the circuit and repetition count.
2. Hash circuits can be expensive, especially when every credential bit has a separate commitment.
3. Reusing the same public credential root can make presentations linkable.
4. The non-interactive version relies on the Fiat–Shamir transform and the random oracle model.
5. The primitive does not provide revocation, expiry or replay protection by itself.
6. Multiple logically related predicates can reveal information over time.

7. Weak or reused randomness can break commitment hiding or MPC privacy.
8. A production implementation must use canonical serialization and strict domain separation.
9. Side-channel security is not provided by the mathematical proof.

9 Future Work

Important next steps include:

- a complete implementation of proof creation and verification;
- deterministic Boolean circuit generation;
- a small language that compiles credential predicates into circuits;
- proof-size and performance benchmarks;
- optimized commitment layouts and hash circuits;
- compressed random tapes and view openings;
- unlinkable credential presentation;
- a complete issuer–holder–verifier protocol;
- revocation, expiry and nonce-based replay protection;
- independent security review.

10 Conclusion

Mayanna–HashMin Primitives provides a simple way to prove predicates over small issuer-authenticated credential attributes without revealing the credential value.

The issuer commits to each credential bit with fresh randomness, places the commitments in a Merkle tree and signs the root.

The holder later proves that the private bits correctly open those commitments and that the same bits satisfy a public predicate.

The proof is created by simulating three MPC parties, committing to their views and opening two challenged views in every repetition. Fiat–Shamir can make the proof non-interactive.

The verifier learns that the predicate is true, but does not learn the comparison prefix, decisive bit or any other credential bit.

The construction is intentionally modest. It is not a succinct proof system and does not aim to replace optimized general-purpose zero-knowledge systems. Its purpose is to provide a readable and audit-oriented primitive for small credential predicates.

References

[1] Y. Ishai, E. Kushilevitz, R. Ostrovsky, and A. Sahai. Zero-Knowledge from Secure Multiparty Computation. In *Proceedings of the 39th Annual ACM Symposium on Theory of Computing*, pages 21–30, 2007.

[2] Y. Ishai, E. Kushilevitz, R. Ostrovsky, and A. Sahai. Zero-Knowledge Proofs from Secure Multiparty Computation. *SIAM Journal on Computing*, 39(3):1121–1152, 2009.

[3] I. Giacomelli, J. Madsen, and C. Orlandi. ZKBoo: Faster Zero-Knowledge for Boolean Circuits. In *25th USENIX Security Symposium*, pages 1069–1083, 2016.

[4] A. Fiat and A. Shamir. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology—CRYPTO '86*, pages 186–194, 1986.

[5] R. C. Merkle. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology—CRYPTO '87*, pages 369–378, 1987.

[6] K. Chalkias, S. Cohen, K. Lewi, F. Moezinia, and Y. Romailler. HashWires: Hyper-Efficient Credential-Based Range Proofs. *Proceedings on Privacy Enhancing Technologies*, 2021(2):456–477, 2021.

[7] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. Bulletproofs: Short Proofs for Confidential Transactions and More. In *2018 IEEE Symposium on Security and Privacy*, pages 315–334, 2018.

[8] J. Groth. On the Size of Pairing-Based Non-Interactive Arguments. In *Advances in Cryptology—EUROCRYPT 2016*, pages 305–326, 2016.